

03 - Next.js Architecture & Performance

Production-minded frontend structure with minimal unnecessary re-rendering

GROW MY THERAPY - STAGE 2 SELECTION ASSIGNMENT

1. Architecture recommendation

Use a static, content-driven Next.js App Router site. Most of this homepage can be Server Components. Only the mobile menu and FAQ interaction need client-side JavaScript. This keeps the page fast, reduces hydration cost, and naturally avoids unnecessary React re-renders.

2. Suggested folder structure

```
src/  
  app/  
    layout.tsx  
    page.tsx  
    globals.css  
    sitemap.ts  
    robots.ts  
  components/  
    layout/  
      Header.tsx  
      MobileNav.tsx    // client only  
      Footer.tsx  
    sections/  
      HeroSection.tsx  
      EmpathySection.tsx  
      ServiceGrid.tsx  
      ApproachSection.tsx  
      AboutMayaSection.tsx  
      OfficeSection.tsx  
      FaqSection.tsx  
      FinalCtaSection.tsx  
    ui/  
      Button.tsx  
      SectionHeading.tsx  
      ResponsiveImage.tsx  
  content/  
    maya.ts  
    navigation.ts  
  lib/  
    seo.ts  
    cn.ts  
  public/  
    images/  
      maya/  
      office/  
      services/
```

3. Component rules

Rule	Reason
Server Components by default	Static content should not hydrate on the client.
Use "use client" only for MobileNav/FAQ if necessary	Limits JS and rerender scope.
Keep content in typed objects	Easy proofreading, profile-source auditing, copy replacement and consistent rendering.
No global state library	There is no application state that justifies Redux/Zustand.
No useEffect for static layout	Avoids client waterfalls and accidental rerenders.
Avoid premature React.memo	Memoization adds complexity; use it only if profiling reveals a real problem.
Stable component boundaries	Each homepage section should be reusable and independently testable.
Use next/image	Responsive image sizes, modern formats and layout stability.

4. Content model example

```
export const maya = {
  name: "Dr. Maya Reynolds, PsyD",
  role: "Licensed Clinical Psychologist",
  location: "Santa Monica, California",
  serviceModes: ["In-person", "Secure telehealth in California"],
  specialties: ["Anxiety", "Panic", "Trauma", "Burnout", "Perfectionism"],
  modalities: ["CBT", "EMDR", "Mindfulness-based practices", "Body-oriented techniques"],
} as const;
```

5. Re-render strategy

- Static server-rendered sections do not rerender in the browser after hydration because they are not client components.
- Keep the FAQ state local to FaqSection. Opening one FAQ should not update the rest of the page.
- Keep mobile menu state local to MobileNav. Do not lift it to layout or a global context.
- Do not place a client provider around the whole application unless absolutely required.
- Prefer CSS hover/focus animations instead of JavaScript animation state.
- If using a motion library, load it only in the component that needs it and respect reduced-motion preferences.
- Use stable image dimensions/aspect ratios to prevent cumulative layout shift.

6. Tailwind design-token setup

Define the redesign palette and typography as reusable tokens rather than scattered hex values. If using Tailwind v4, expose the values through CSS variables and theme utilities; if using another stable Tailwind version, map the same tokens in the supported config format.

```
@theme {
  --color-primary: #24352F;
  --color-secondary: #E9DED0;
  --color-accent: #B8795B;
  --color-surface: #FAF8F4;
  --color-ink: #242723;
}

:root {
  --section-y: clamp(3rem, 7vw, 7.5rem);
  --content-max: 76rem;
}
```

7. Performance budget

Metric	Target / practice
LCP	Aim under ~2.5s on a normal mobile connection. Preload only the hero image/font that truly matters.
CLS	Near zero: set width/height or aspect-ratio for every image.
JS	Keep client JS minimal; avoid sliders/carousels unless the reference requires them.
Fonts	Use next/font or self-hosted licensed fonts; limit weights/styles.
Images	AVIF/WebP where supported; responsive sizes; avoid multi-megabyte originals.
Third-party scripts	None by default. Add analytics only if useful; do not compromise the assignment with unnecessary scripts.

8. Accessibility implementation

- Semantic landmarks: header, nav, main, sections with labelled headings, footer.
- Heading order should be H1 then meaningful H2/H3 levels; do not choose heading tags based only on size.
- Use button for actions and anchor for navigation.
- Mobile navigation must expose aria-expanded and be fully keyboard controllable.
- Use native details/summary for FAQs if it visually fits; otherwise implement accordion with correct ARIA relationships.
- Minimum touch target around 44x44 CSS pixels.
- Add skip-to-content link and strong focus-visible styling.

9. Testing stack

Tool	Use
TypeScript	Strict component/content typing and compile-time safety.
ESLint	Catch common Next/React issues.
Playwright	A few high-value checks: homepage loads, mobile menu works, FAQ works, no overflow, CTA links resolve.
Lighthouse	Performance, accessibility, best practices, SEO snapshot.
axe / browser accessibility tools	Keyboard/contrast/ARIA inspection.
Visual comparison	Reference screenshots vs clone at fixed widths; inspect after each major pass.